

VDF Assignment-2

VLSI Design Flow

Verilog Modelling, Simulation, Code Coverage and Logic Synthesis

Question 1 — Proof of Tool Execution

All work was done on Ubuntu 22.04 using the open-source flow: Icarus Verilog 11.0 for simulation, GTKWave 3.3.104 for waveform viewing, Covered 20090802 for code coverage, and Yosys 0.9 with Berkeley ABC for synthesis.

```
aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Assignment-2
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ iverilog -Wall -o test_gray fsm_gray.v tb_fsm.v
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ vvp test_gray
VCD info: dumpfile fsm.vcd opened for output.
--- reset released, FSM should be in S1 ---
[21000] S1 in=00 out=01 -> S1
[31000] S1 in=01 out=00 -> S1
[41000] S1 in=10 out=01 -> S2
[51000] S2 in=10 out=10 -> S2
[61000] S2 in=00 out=11 -> S1
[71000] S1 in=10 out=01 -> S2
[81000] S2 in=01 out=11 -> S3
[91000] S3 in=10 out=10 -> S3
[101000] S3 in=00 out=00 -> S1
[111000] S1 in=11 out=00 -> S3
[121000] S3 in=11 out=00 -> S4
[131000] S4 in=11 out=11 -> S4
[141000] S4 in=00 out=11 -> S3
[151000] S3 in=01 out=01 -> S5
[161000] S5 in=00 out=00 -> S5
[171000] S5 in=11 out=11 -> S5
[181000] S5 in=10 out=00 -> S3
[191000] S3 in=11 out=00 -> S4
[201000] S4 in=10 out=01 -> S5
[211000] S5 in=01 out=00 -> S1
[221000] S1 in=10 out=01 -> S2
[231000] S2 in=11 out=10 -> S4
[241000] S4 in=01 out=01 -> S5
[251000] S5 in=01 out=00 -> S1
--- synchronous reset re-applied, FSM back in S1 ---
[281000] S1 in=10 out=01 -> S2

TEST PASSED : all 25 checks matched the reference model
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$
```

Figure 1: Icarus Verilog — compilation and simulation of the gray-encoded design. All 25 checked vectors match the reference model.

```
aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Assignment-2
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ iverilog -Wall -o test_cov fsm_gray.v tb_cov.v
vvp test_cov
covered score -t tb_cov -v tb_cov.v -v fsm_gray.v -vcd fsm_cov.vcd -o fsm.cdd
VCD info: dumpfile fsm_cov.vcd opened for output.
Coverage stimulus complete.

Covered covered-20090802 -- Verilog Code Coverage Utility
Written by Trevor Williams (phase1geo@gmail.com)
Copyright 2006-2010
Freely distributable under the GPL license

Reading design...
Parsing file 'tb_cov.v'
Parsing file 'fsm_gray.v'

Checking for race conditions...

Scoring VCD dumpfile fsm_cov.vcd...

*** Scoring completed successfully! ***

Dynamic memory allocated: 358211 bytes

aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$
```

Figure 2: Covered — scoring the design against the VCD dumpfile.

```

aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Assignment-2
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ yosys -s syn_gray_lib.y | tail -30
<suppressed ~2 debug messages>

14. Printing statistics.

=== fsm_gray ===

Number of wires:          57
Number of wire bits:      63
Number of public wires:   5
Number of public wire bits: 9
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          59
  AND2X1                   2
  DFFPOSX1                 3
  INVX1                    8
  NAND2X1                  17
  NOR2X1                   24
  OR2X1                    5

Chip area for module '\fsm_gray': 147.000000

15. Executing Verilog backend.
Dumping module '\fsm_gray'.

End of script. Logfile hash: b7a3edf48e
CPU: user 0.09s system 0.00s, MEM: 17.50 MB total, 11.80 MB resident
Yosys 0.9 (git sha1 1979e0b)
Time spent: 14% 10x opt_clean (0 sec), 14% 10x opt_merge (0 sec), ...
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$

```

Figure 3: Yosys — synthesis of the gray-encoded design mapped to the technology library. Chip area 147.0.

```

aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Assignment-2
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ yosys -s syn_onehot_lib.y | tail -30
<suppressed ~2 debug messages>

14. Printing statistics.

=== fsm_onehot ===

Number of wires:          68
Number of wire bits:      78
Number of public wires:   5
Number of public wire bits: 11
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          74
  AND2X1                   4
  DFFPOSX1                 5
  INVX1                    7
  NAND2X1                  19
  NOR2X1                   33
  OR2X1                    6

Chip area for module '\fsm_onehot': 201.000000

15. Executing Verilog backend.
Dumping module '\fsm_onehot'.

End of script. Logfile hash: 34b248d9ad
CPU: user 0.10s system 0.00s, MEM: 17.80 MB total, 12.17 MB resident
Yosys 0.9 (git sha1 1979e0b)
Time spent: 15% 10x opt_expr (0 sec), 14% 10x opt_merge (0 sec), ...
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$

```

Figure 4: Yosys — synthesis of the one-hot design mapped to the same library. Chip area 201.0.

Question 2(a) — Verilog Model with Gray Encoding

State transition table

Read from the given state diagram; each entry is next-state / output. The machine is Mealy, since outputs are labelled on the arcs rather than inside the state bubbles. Every state has four outgoing arcs and all 20 labels are accounted for.

State	in = 00	in = 01	in = 10	in = 11
S1 (000)	S1 / 01	S1 / 00	S2 / 01	S3 / 00
S2 (001)	S1 / 11	S3 / 11	S2 / 10	S4 / 10
S3 (011)	S1 / 00	S5 / 01	S3 / 10	S4 / 00
S4 (010)	S3 / 11	S5 / 01	S5 / 01	S4 / 11

S5 (110)	S5 / 00	S1 / 00	S3 / 00	S5 / 11
----------	---------	---------	---------	---------

Choice of gray encoding

Five states need three bits. Codes are assigned so consecutive ones differ in a single bit: S1 = 000, S2 = 001, S3 = 011, S4 = 010, S5 = 110. Only one flip-flop toggles between adjacent states, lowering switching activity and removing glitches caused by several bits changing at slightly different times. Transitions that skip states, such as S1 to S3, still flip two bits.

Three codes (100, 101, 111) are unused; a default branch maps them back to S1 so the design cannot lock up. Both recommended practices for avoiding latch inference are applied: a default case covers every path, and next_state and out are given default values at the top of the combinational block. The sensitivity list is @(*). Part (e) confirms this worked, as the netlist contains no latches. The sequential block uses non-blocking assignment, the combinational block blocking.

Source code — fsm_gray.v

```
`timescale 1ns/1ps
//=====================================================
// VDF Assignment-2, Q2(a)
// Mealy FSM, 5 states, 2-bit input, 2-bit output
// State encoding : GRAY (3 bits)
// S1 = 000, S2 = 001, S3 = 011, S4 = 010, S5 = 110
// consecutive codes differ in exactly one bit
//=====================================================

module fsm_gray (
    input wire      clk,
    input wire      rst,    // synchronous, active high -> S1
    input wire [1:0] in,
    output reg [1:0] out
);

    // ----- gray state codes -----
    localparam [2:0] S1 = 3'b000,
                   S2 = 3'b001,
                   S3 = 3'b011,
                   S4 = 3'b010,
                   S5 = 3'b110;

    (* fsm_encoding = "none" *) reg [2:0] cur_state;
    reg [2:0] next_state;

    // ----- state register -----
    always @(posedge clk) begin
        if (rst)
            cur_state <= S1;
        else
            cur_state <= next_state;
    end

    // ----- next state + output logic -----
    // Mealy: output is a function of (cur_state, in)
    always @(*) begin
        // safe defaults (avoid latches / unknown propagation)
        next_state = S1;
        out = 2'b00;

        case (cur_state)

            S1: case (in)
                2'b00: begin next_state = S1; out = 2'b01; end
                2'b01: begin next_state = S1; out = 2'b00; end
                2'b10: begin next_state = S2; out = 2'b01; end
                2'b11: begin next_state = S3; out = 2'b00; end
            endcase

            S2: case (in)
                2'b00: begin next_state = S1; out = 2'b11; end
                2'b01: begin next_state = S3; out = 2'b11; end
                2'b10: begin next_state = S2; out = 2'b10; end
                2'b11: begin next_state = S4; out = 2'b10; end
            endcase

            S3: case (in)
                2'b00: begin next_state = S1; out = 2'b00; end
                2'b01: begin next_state = S5; out = 2'b01; end
                2'b10: begin next_state = S3; out = 2'b10; end
                2'b11: begin next_state = S4; out = 2'b00; end
            endcase

            S4: case (in)
                2'b00: begin next_state = S3; out = 2'b11; end
                2'b01: begin next_state = S5; out = 2'b01; end
                2'b10: begin next_state = S5; out = 2'b01; end
                2'b11: begin next_state = S4; out = 2'b11; end
            endcase

            S5: case (in)
```

```

                2'b00: begin next_state = S5; out = 2'b00; end
                2'b01: begin next_state = S1; out = 2'b00; end
                2'b10: begin next_state = S3; out = 2'b00; end
                2'b11: begin next_state = S5; out = 2'b11; end
            endcase

            // unused gray codes 100, 101, 111 -> recover to S1
            default: begin next_state = S1; out = 2'b00; end

        endcase
    end
endmodule

```

Question 2(b) — Testbench

The testbench is self-checking rather than a plain stimulus generator, and has three parts:

- A directed walk of 24 vectors exercising all 20 arcs, followed by a reset applied from a non-initial state.
- A reference model written from the state diagram, using state numbers 1 to 5. Being encoding-independent, it validates both the gray and one-hot versions unchanged.
- A per-vector comparison; mismatches are counted and printed, and the run ends with TEST PASSED or TEST FAILED.

Since the design is Mealy, inputs are applied on the falling edge and outputs sampled just before the next rising edge, letting the combinational output settle while still checking it in the same cycle. Compiling with -DONEHOT selects the one-hot module.

Source code — tb_fsm.v

```

//=====
// VDF Assignment-2, Q2(b)
// Self-checking testbench for the 5-state Mealy FSM.
//
// The stimulus is a directed walk that exercises ALL 20 arcs
// of the state diagram (5 states x 4 input combinations), plus
// the reset behaviour. A golden reference model (written from
// the state diagram, independent of the DUT's encoding) predicts
// the output every cycle and any mismatch is reported.
//
// Compile (gray version) : iverilog -Wall -o test_fsm_gray.v tb_fsm.v
// Compile (one-hot version): iverilog -Wall -DONEHOT -o test_fsm_onehot.v tb_fsm.v
// Run : vvp test
// View : gtkwave fsm.vcd
//=====

`timescale 1ns/1ps

module tb_fsm;

    reg        clk = 1'b0;
    reg        rst;
    reg [1:0] in;
    wire [1:0] out;

    integer errors = 0;
    integer i;

    // ----- DUT -----
    `ifdef ONEHOT
        fsm_onehot dut (.clk(clk), .rst(rst), .in(in), .out(out));
    `else
        fsm_gray dut (.clk(clk), .rst(rst), .in(in), .out(out));
    `endif

    // ----- clock : 10 ns period -----
    always #5 clk = ~clk;

    // ----- golden reference model -----
    // states numbered 1..5, encoding-independent
    integer ref_state;

    // returns {next_state[2:0], expected_out[1:0]}
    function [4:0] model;
        input integer s;
        input [1:0] stim;
        begin
            model = 5'b0;
            case (s)
            1: case (stim)
                2'b00: model = {3'd1, 2'b01};
                2'b01: model = {3'd1, 2'b00};
                2'b10: model = {3'd2, 2'b01};
                2'b11: model = {3'd3, 2'b00};
            endcase
            2: case (stim)
                2'b00: model = {3'd1, 2'b11};
                2'b01: model = {3'd3, 2'b11};
                2'b10: model = {3'd2, 2'b10};
            endcase
        end
    end

```

```

        2'b11: model = {3'd4, 2'b10};
    endcase
3: case (stim)
    2'b00: model = {3'd1, 2'b00};
    2'b01: model = {3'd5, 2'b01};
    2'b10: model = {3'd3, 2'b10};
    2'b11: model = {3'd4, 2'b00};
endcase
4: case (stim)
    2'b00: model = {3'd3, 2'b11};
    2'b01: model = {3'd5, 2'b01};
    2'b10: model = {3'd5, 2'b01};
    2'b11: model = {3'd4, 2'b11};
endcase
5: case (stim)
    2'b00: model = {3'd5, 2'b00};
    2'b01: model = {3'd1, 2'b00};
    2'b10: model = {3'd3, 2'b00};
    2'b11: model = {3'd5, 2'b11};
endcase
endcase
end
endfunction

// ----- one stimulus vector -----
task apply;
    input [1:0] stim;
    reg [4:0] pred;
    begin
        @(negedge clk);
        in = stim;
        pred = model(ref_state, stim);
        #1; // let combinational output settle
        if (out != pred[1:0]) begin
            errors = errors + 1;
            $display("[%0t] MISMATCH: S%d in=%b -> out=%b (expected %b)",
                $time, ref_state, stim, out, pred[1:0]);
        end else begin
            $display("[%0t] S%d in=%b out=%b -> S%d",
                $time, ref_state, stim, out, pred[4:2]);
        end
        @(posedge clk);
        #1;
        ref_state = pred[4:2]; // reference follows the DUT
    end
endtask

// ----- directed walk covering all 20 arcs -----
// S1:00 S1:01 S1:10 S2:10 S2:00 (S1:10) S2:01 S3:10 S3:00 S1:11
// S3:11 S4:11 S4:00 S3:01 S5:00 S5:11 S5:10 (S3:11) S4:10 S5:01
// (S1:10) S2:11 S4:01 S5:01
reg [1:0] walk [0:23];
initial begin
    walk[0]=2'b00; walk[1]=2'b01; walk[2]=2'b10; walk[3]=2'b10;
    walk[4]=2'b00; walk[5]=2'b10; walk[6]=2'b01; walk[7]=2'b10;
    walk[8]=2'b00; walk[9]=2'b11; walk[10]=2'b11; walk[11]=2'b11;
    walk[12]=2'b00; walk[13]=2'b01; walk[14]=2'b00; walk[15]=2'b11;
    walk[16]=2'b10; walk[17]=2'b11; walk[18]=2'b10; walk[19]=2'b01;
    walk[20]=2'b10; walk[21]=2'b11; walk[22]=2'b01; walk[23]=2'b01;
end

// ----- main stimulus -----
initial begin
    $dumpfile("fsm.vcd");
    $dumpvars(0, tb_fsm);

    // reset
    rst = 1'b1;
    in = 2'b00;
    ref_state = 1;
    @(posedge clk);
    @(posedge clk);
    #1 rst = 1'b0;
    $display("--- reset released, FSM should be in S1 ---");

    // walk through every arc
    for (i = 0; i < 24; i = i + 1)
        apply(walk[i]);

    // check reset from a non-initial state
    @(negedge clk); in = 2'b11; @(posedge clk); // move away from S1
    @(negedge clk); rst = 1'b1;
    @(posedge clk); #1 rst = 1'b0;
    ref_state = 1;
    $display("--- synchronous reset re-applied, FSM back in S1 ---");
    apply(2'b10); // must behave like S1 again

    if (errors == 0)
        $display("\nTEST PASSED : all %0d checks matched the reference model", 25);
    else
        $display("\nTEST FAILED : %0d mismatches", errors);

    #20 $finish;
end
endmodule

```

Question 2(c) — Simulation Waveform and Correctness

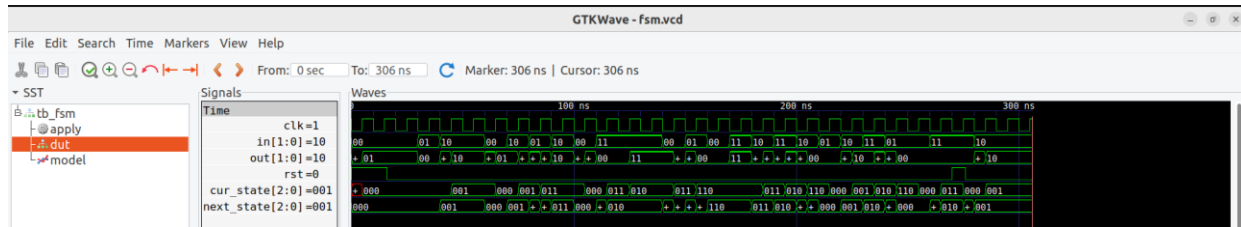


Figure 5: Complete simulation waveform, 0 to 306 ns. Both reset pulses are visible and `cur_state` takes only the five legal gray codes.

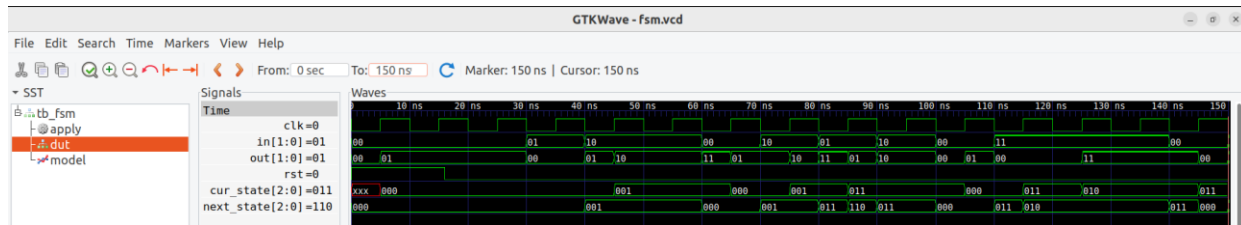


Figure 6: Zoomed waveform, 0 to 150 ns, with individual transitions readable. Note `cur_state = xxx` before the first clock edge.

Explanation of correctness

1. Reset. `cur_state` is xxx until the first rising edge, since the flip-flops are uninitialised at time zero. With `rst` high, that edge loads 000, which is S1. A later reset pulse forces the machine back to 000 from a different state, so the reset is unconditional.
2. Legal codes only. `cur_state` takes just 000, 001, 011, 010 and 110 across the whole run. The three unused codes never appear, so no illegal state is reachable.
3. State register. At every rising edge `cur_state` takes the value `next_state` held just before it, giving a consistent one-cycle offset between the two rows.
4. Mealy behaviour. `out` changes with `in`, not only at clock edges, visible at several points mid-cycle. This matches the diagram, where outputs are written on the arcs and so depend on both state and input.

Figure 1 additionally shows all 25 vectors matching the reference model. Since the walk covers all 20 transitions, this checks the entire transition table, not a sample of it.

Question 2(d) — Line Coverage Report

Commands used:

```
iverilog -Wall -o test_cov fsm_gray.v tb_cov.v
vvp test_cov
covered score -t tb_cov -v tb_cov.v -v fsm_gray.v -vcd fsm_cov.vcd -o fsm.cdd
covered report -d v fsm.cdd
```

The available Covered release is dated 2009 and its parser rejects the reference-model function in `tb_fsm.v`, so a stimulus-only testbench, `tb_cov.v`, was used for scoring. It applies the same 24-vector walk and reset sequence without functions or tasks, leaving the coverage figures for `fsm_gray.v` unchanged.

```

aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Assignment-2
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ covered report -d v fsm.cdd | head -45

Covered covered-20090802 -- Verilog Code Coverage Utility
Written by Trevor Williams (phase1geo@gmail.com)
Copyright 2006-2010
Freely distributable under the GPL license

:::
::
:: Covered -- Verilog Coverage Verbose Report ::
::
:::

-----
GENERAL INFORMATION
-----
* Report generated from CDD file : fsm.cdd
* Reported by : Module

-----
LINE COVERAGE RESULTS
-----
Module/Task/Function  Filename      Hit/ Miss/Total  Percent hit
-----
$root                NA           0/  0/  0       100%
tb_cov               tb_cov.v     63/  0/ 63       100%
fsm_gray             fsm_gray.v   49/  0/ 49       100%
-----
Accumulated          112/  0/ 112     100%

-----
TOGGLE COVERAGE RESULTS
-----
Module/Task/Function  Filename      Hit/ Miss/Total  Percent hit  Toggle 0 -> 1  Percent hit  Toggle 1 -> 0  Percent hit
-----
$root                NA           0/  0/  0       100%           0/  0/  0       100%
tb_cov               tb_cov.v     6/  0/  6       100%           6/  0/  6       100%
fsm_gray             fsm_gray.v   12/  0/ 12       100%          12/  0/ 12       100%
-----
Accumulated          18/  0/ 18       100%          18/  0/ 18       100%

```

Figure 7: Line coverage and toggle coverage results.

```

aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Assignment-2
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ covered report -d v fsm.cdd | sed -n '45,110p'

-----
COMBINATIONAL LOGIC COVERAGE RESULTS
-----
Module/Task/Function  Filename      Hit/ Miss/Total  Percent hit  Logic Combinations
-----
$root                NA           0/  0/  0       100%           0/  0/  0       100%
tb_cov               tb_cov.v     33/  0/ 33       100%           33/  0/ 33       100%
fsm_gray             fsm_gray.v   18/  0/ 18       100%           18/  0/ 18       100%
-----
Accumulated          51/  0/ 51       100%

-----
FINITE STATE MACHINE COVERAGE RESULTS
-----
Module/Task/Function  Filename      Hit/ Miss/Total  Percent Hit  State  Arc  Hit/ Miss/Total  Percent hit
-----
$root                NA           0/  0/  0       100%           0/  0/  0       100%
tb_cov               tb_cov.v     0/  0/  0       100%           0/  0/  0       100%
fsm_gray             fsm_gray.v   0/  0/  0       100%           0/  0/  0       100%
-----
Accumulated          0/  0/  0       100%           0/  0/  0       100%

```

Figure 8: Combinational logic coverage and finite state machine coverage results.

Explanation of the report

Line coverage. 49 of 49 executable lines hit in fsm_gray.v, and 112 of 112 across the design. The walk visits all five states and applies all four inputs in each, so every branch of every case statement executes.

The default branch. The default covering the three unused codes is unreachable, so it might be expected to show as a miss. It does not: the elaborator finds the case statement full and treats the branch as unreachable, so it never enters the denominator. The 100 percent figure therefore means every reachable line ran, not that the defensive code was exercised.

Toggle coverage. 12 of 12 bits toggle in each direction, confirming all three state bits and both bits of the input and output buses were driven to both values.

Combinational logic coverage. 18 of 18 combinations hit. This is stronger again, since it checks whether each sub-expression took the values needed to influence the result rather than merely whether signals were active.

FSM coverage. The section shows 0 of 0 states and arcs. Covered only measures these when told which signal holds the state, via the -F option, which was not supplied. This matters because arc coverage would be the most relevant metric

here, directly confirming all 20 transitions were traversed. That confirmation comes instead from the self-checking testbench in part (b).

Interpretation. The stimulus fully exercises the design at line, toggle and logic-combination level. Coverage measures what the stimulus reached, though, not whether results were correct — a testbench running the same walk and checking nothing would score identically. Correctness comes from the reference-model comparison. Coverage shows the verification had the chance to find a bug, not that none exists.

Source code — tb_cov.v

```
//=====
// VDF Assignment-2, Q2(d)
// Stimulus-only testbench used for COVERED line-coverage scoring.
//
// It applies exactly the same 24-vector directed walk as tb_fsm.v
// (all 20 arcs of the state diagram), plus the reset sequence, but
// contains no functions and no tasks, because the Covered 2009
// parser cannot handle the self-checking reference model used in
// tb_fsm.v. Since line coverage is measured on fsm_gray.v and the
// stimulus is identical, the coverage figures are unaffected.
//
// iverilog -Wall -o test_cov fsm_gray.v tb_cov.v
// vvp test_cov
// covered_score -t tb_cov -v tb_cov.v -v fsm_gray.v -vcd fsm_cov.vcd -o fsm.cdd
// covered_report -d v fsm.cdd
//=====

`timescale 1ns/1ps

module tb_cov;

    reg        clk = 1'b0;
    reg        rst;
    reg [1:0]  in;
    wire [1:0] out;

    fsm_gray dut (.clk(clk), .rst(rst), .in(in), .out(out));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("fsm_cov.vcd");
        $dumpvars(0, tb_cov);

        // ---- reset into S1 ----
        rst = 1'b1;
        in  = 2'b00;
        @(posedge clk);
        @(posedge clk);
        #1 rst = 1'b0;

        // ---- directed walk covering all 20 arcs ----
        @(negedge clk); in = 2'b00; // S1 / 00 -> S1
        @(negedge clk); in = 2'b01; // S1 / 01 -> S1
        @(negedge clk); in = 2'b10; // S1 / 10 -> S2
        @(negedge clk); in = 2'b10; // S2 / 10 -> S2
        @(negedge clk); in = 2'b00; // S2 / 00 -> S1
        @(negedge clk); in = 2'b10; // S1 / 10 -> S2
        @(negedge clk); in = 2'b01; // S2 / 01 -> S3
        @(negedge clk); in = 2'b10; // S3 / 10 -> S3
        @(negedge clk); in = 2'b00; // S3 / 00 -> S1
        @(negedge clk); in = 2'b11; // S1 / 11 -> S3
        @(negedge clk); in = 2'b11; // S3 / 11 -> S4
        @(negedge clk); in = 2'b11; // S4 / 11 -> S4
        @(negedge clk); in = 2'b00; // S4 / 00 -> S3
        @(negedge clk); in = 2'b01; // S3 / 01 -> S5
        @(negedge clk); in = 2'b00; // S5 / 00 -> S5
        @(negedge clk); in = 2'b11; // S5 / 11 -> S5
        @(negedge clk); in = 2'b10; // S5 / 10 -> S3
        @(negedge clk); in = 2'b11; // S3 / 11 -> S4
        @(negedge clk); in = 2'b10; // S4 / 10 -> S5
        @(negedge clk); in = 2'b01; // S5 / 01 -> S1
        @(negedge clk); in = 2'b10; // S1 / 10 -> S2
        @(negedge clk); in = 2'b11; // S2 / 11 -> S4
        @(negedge clk); in = 2'b01; // S4 / 01 -> S5
        @(negedge clk); in = 2'b01; // S5 / 01 -> S1

        // ---- reset applied from a non-initial state ----
        @(negedge clk); in = 2'b11;
        @(negedge clk); rst = 1'b1;
        @(posedge clk); #1 rst = 1'b0;
        @(negedge clk); in = 2'b10;

        @(negedge clk);
        #20 $display("Coverage stimulus complete.");
        $finish;
    end

endmodule
```


Question 2(e) — Synthesis of the Gray-Encoded Design

Technology library

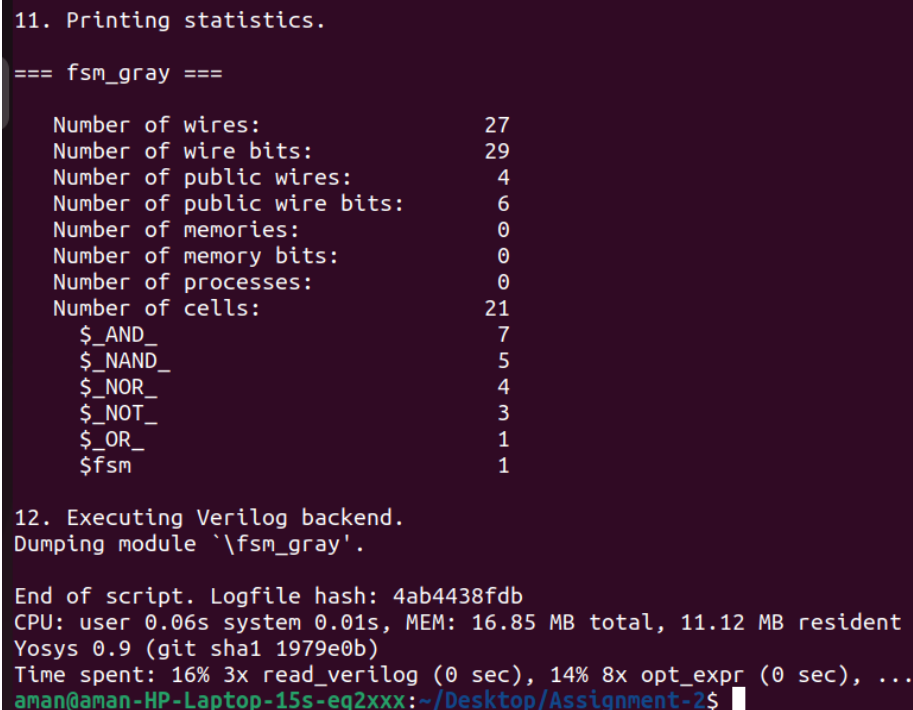
A small Liberty library, `simple.lib`, was written for the mapping, following the standard structure: a header with units and nominal PVT conditions, then cells carrying an area and a pin list, each pin carrying direction, capacitance, a function expression for outputs, and timing arcs. It provides an inverter (area 1), buffer (2), two-input NAND and NOR (2 each), AND and OR (3 each), XOR (5) and a positive-edge D flip-flop (12). The areas scale as in a real library: NAND and NOR are single CMOS stages and so cheaper than AND and OR, which are a NAND or NOR plus an inverter, and the flip-flop costs several times any gate. The same library is used in part (f).

Synthesis script (`syn_gray_lib.y`):

```
read_verilog fsm_gray.v
hierarchy -check -top fsm_gray
proc; opt
fsm; opt
memory; opt
techmap; opt
dfflibmap -liberty simple.lib
abc -liberty simple.lib
opt_clean
stat -liberty simple.lib
write_verilog -noattr fsm_gray_netlist_lib.v
```

Preserving the encoding

Yosys detects a state register by default, extracts it into an abstract FSM object and re-encodes it. On the first attempt the log reported "Found FSM state register `fsm_gray.cur_state`" and then "mapping auto encoding to one-hot for this FSM" — the gray encoding was silently discarded, and the netlist came out containing a `$fsm` cell and no flip-flops. Any comparison with part (f) would then be meaningless, since both sources would map to the same internal encoding.



```
11. Printing statistics.

=== fsm_gray ===

Number of wires:                27
Number of wire bits:            29
Number of public wires:         4
Number of public wire bits:     6
Number of memories:             0
Number of memory bits:          0
Number of processes:            0
Number of cells:                21
  $_AND_                        7
  $_NAND_                       5
  $_NOR_                        4
  $_NOT_                        3
  $_OR_                         1
  $fsm                         1

12. Executing Verilog backend.
Dumping module `fsm_gray'.

End of script. Logfile hash: 4ab4438fdb
CPU: user 0.06s system 0.01s, MEM: 16.85 MB total, 11.12 MB resident
Yosys 0.9 (git sha1 1979e0b)
Time spent: 16% 3x read_verilog (0 sec), 14% 8x opt_expr (0 sec), ...
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$
```

Figure 9: First synthesis attempt, before the fix. The netlist contains a `$fsm` cell and no flip-flops, because Yosys had re-encoded the state register itself.

The fix is the attribute `(* fsm_encoding = "none" *)` on the state register. With it, `FSM_DETECT` reports no FSM found and the chosen encoding survives into the netlist. The results below only appear once the tool is stopped from overriding the source, a reminder that a synthesis result reflects the RTL and the tool settings together.

Synthesised netlist — `fsm_gray_netlist_lib.v`

```
/* Generated by Yosys 0.9 (git sha1 1979e0b) */
```

```

module fsm_gray(clk, rst, in, out);
  wire [2:0] _000_;
  wire _001_; wire _002_; wire _003_; wire _004_; wire _005_; wire _006_;
  wire _007_; wire _008_; wire _009_; wire _010_; wire _011_; wire _012_;
  wire _013_; wire _014_; wire _015_; wire _016_; wire _017_; wire _018_;
  wire _019_; wire _020_; wire _021_; wire _022_; wire _023_; wire _024_;
  wire _025_; wire _026_; wire _027_; wire _028_; wire _029_; wire _030_;
  wire _031_; wire _032_; wire _033_; wire _034_; wire _035_; wire _036_;
  wire _037_; wire _038_; wire _039_; wire _040_; wire _041_; wire _042_;
  wire _043_; wire _044_; wire _045_; wire _046_; wire _047_; wire _048_;
  wire _049_; wire _050_; wire _051_;
  input clk;
  wire [2:0] cur_state;
  input [1:0] in;
  output [1:0] out;
  input rst;

  // ----- shared input / state decode -----
  INVX1 _052_ ( .A(in[0]), .Y(_001_) );
  INVX1 _053_ ( .A(in[1]), .Y(_002_) );
  INVX1 _054_ ( .A(cur_state[0]), .Y(_003_) );
  INVX1 _055_ ( .A(cur_state[2]), .Y(_004_) );
  INVX1 _056_ ( .A(cur_state[1]), .Y(_005_) );
  NOR2X1 _057_ ( .A(in[0]), .B(in[1]), .Y(_006_) );
  OR2X1 _058_ ( .A(in[0]), .B(in[1]), .Y(_007_) );
  NAND2X1 _059_ ( .A(in[0]), .B(in[1]), .Y(_008_) );
  INVX1 _060_ ( .A(_008_), .Y(_009_) );
  NAND2X1 _061_ ( .A(_007_), .B(_008_), .Y(_010_) );
  INVX1 _062_ ( .A(_010_), .Y(_011_) );
  NOR2X1 _063_ ( .A(_003_), .B(cur_state[2]), .Y(_012_) );
  AND2X1 _064_ ( .A(_005_), .B(_012_), .Y(_013_) );
  NAND2X1 _065_ ( .A(_001_), .B(in[1]), .Y(_014_) );
  NAND2X1 _066_ ( .A(cur_state[1]), .B(_014_), .Y(_015_) );
  NAND2X1 _067_ ( .A(_012_), .B(_015_), .Y(_016_) );
  OR2X1 _068_ ( .A(_010_), .B(_016_), .Y(_017_) );
  NOR2X1 _069_ ( .A(cur_state[0]), .B(cur_state[2]), .Y(_018_) );
  NAND2X1 _070_ ( .A(_005_), .B(_018_), .Y(_019_) );
  NOR2X1 _071_ ( .A(_002_), .B(_019_), .Y(_020_) );
  NOR2X1 _072_ ( .A(cur_state[0]), .B(_005_), .Y(_021_) );
  NAND2X1 _073_ ( .A(cur_state[2]), .B(_021_), .Y(_022_) );
  NOR2X1 _074_ ( .A(_014_), .B(_022_), .Y(_023_) );
  NAND2X1 _075_ ( .A(_004_), .B(_021_), .Y(_024_) );
  INVX1 _076_ ( .A(_024_), .Y(_025_) );
  NOR2X1 _077_ ( .A(_007_), .B(_024_), .Y(_026_) );

  // ----- next-state bit 0 (with synchronous reset) -----
  NOR2X1 _078_ ( .A(_020_), .B(_026_), .Y(_027_) );
  NAND2X1 _079_ ( .A(_017_), .B(_027_), .Y(_028_) );
  NOR2X1 _080_ ( .A(_023_), .B(_028_), .Y(_029_) );
  NOR2X1 _081_ ( .A(rst), .B(_029_), .Y(_000_[0]) );

  // ----- next-state bit 1 (with synchronous reset) -----
  NOR2X1 _082_ ( .A(_013_), .B(_020_), .Y(_030_) );
  NOR2X1 _083_ ( .A(_001_), .B(_030_), .Y(_031_) );
  NAND2X1 _084_ ( .A(cur_state[1]), .B(_012_), .Y(_032_) );
  NOR2X1 _085_ ( .A(_006_), .B(_032_), .Y(_033_) );
  NOR2X1 _086_ ( .A(_011_), .B(_022_), .Y(_034_) );
  OR2X1 _087_ ( .A(_011_), .B(_022_), .Y(_035_) );
  NOR2X1 _088_ ( .A(_033_), .B(_034_), .Y(_036_) );
  NOR2X1 _089_ ( .A(_023_), .B(_025_), .Y(_037_) );
  NAND2X1 _090_ ( .A(_036_), .B(_037_), .Y(_038_) );
  NOR2X1 _091_ ( .A(_031_), .B(_038_), .Y(_039_) );
  NOR2X1 _092_ ( .A(rst), .B(_039_), .Y(_000_[1]) );

  // ----- next-state bit 2 (with synchronous reset) -----
  NOR2X1 _093_ ( .A(_010_), .B(_024_), .Y(_040_) );
  NOR2X1 _094_ ( .A(_001_), .B(_032_), .Y(_041_) );
  NAND2X1 _095_ ( .A(_002_), .B(_041_), .Y(_042_) );
  NAND2X1 _096_ ( .A(_035_), .B(_042_), .Y(_043_) );
  NOR2X1 _097_ ( .A(_040_), .B(_043_), .Y(_044_) );
  NOR2X1 _098_ ( .A(rst), .B(_044_), .Y(_000_[2]) );

  // ----- Mealy output logic -----
  AND2X1 _099_ ( .A(_002_), .B(_013_), .Y(_045_) );
  NOR2X1 _100_ ( .A(in[0]), .B(_019_), .Y(_046_) );
  NOR2X1 _101_ ( .A(_045_), .B(_046_), .Y(_047_) );
  NAND2X1 _102_ ( .A(_009_), .B(_021_), .Y(_048_) );
  NAND2X1 _103_ ( .A(_024_), .B(_042_), .Y(_049_) );
  NAND2X1 _104_ ( .A(_047_), .B(_048_), .Y(_050_) );
  OR2X1 _105_ ( .A(_049_), .B(_050_), .Y(out[0]) );
  NAND2X1 _106_ ( .A(_016_), .B(_048_), .Y(_051_) );
  OR2X1 _107_ ( .A(_026_), .B(_051_), .Y(out[1]) );

  // ----- state register: 3 library flip-flops -----
  DFFPOSX1 _108_ ( .CK(clk), .D(_000_[0]), .Q(cur_state[0]) );
  DFFPOSX1 _109_ ( .CK(clk), .D(_000_[1]), .Q(cur_state[1]) );
  DFFPOSX1 _110_ ( .CK(clk), .D(_000_[2]), .Q(cur_state[2]) );
endmodule

```

Explanation of the result

The design maps to 59 cells and 147.0 area units: 3 DFFPOSX1, 17 NAND2X1, 24 NOR2X1, 8 INVX1, 5 OR2X1 and 2 AND2X1. The flip-flops account for 36.0 units, leaving 111.0 of combinational logic. Three flip-flops is the minimum for five states, since $\text{ceil}(\log_2 5) = 3$.

The three DFFPOSX1 instances form the state register, each loading from a bit of the next-state vector _000_. The reset is cheap: every next-state bit ends in a NOR2X1 taking rst and the computed value, so asserting rst forces all three bits to 0, giving S1. The whole reset costs three gates.

The gate mix follows the library cost function. NOR2X1 and NAND2X1 dominate at 2 units each, while AND2X1 and OR2X1 at 3 units appear only where unavoidable. Eight inverters generate the complements _001_ to _005_, shared by many downstream gates. This sharing and the bias toward negative-logic cells are typical of mapping against an area objective.

The cells driving out[0] and out[1] take in[0] and in[1] directly, confirming Mealy behaviour at netlist level. No latches appear anywhere, so no branch of the combinational block was left unassigned.

Question 2(f) — One-Hot Encoding and Comparison

The same machine re-coded with one flip-flop per state, exactly one bit high at a time: S1 = 00001, S2 = 00010, S3 = 00100, S4 = 01000, S5 = 10000. The fsm_encoding attribute is applied here too so Yosys preserves it.

Source code — fsm_onehot.v

```
`timescale 1ns/1ps
//=====
// VDF Assignment-2, Q2(f)
// Same FSM as fsm_gray.v, but ONE-HOT encoded (5 bits)
// S1 = 00001, S2 = 00010, S3 = 00100, S4 = 01000, S5 = 10000
//
// NOTE: the (* fsm_encoding = "none" *) attribute stops Yosys'
// 'fsm' pass from re-encoding this register back to binary.
// Without it Yosys recognises the FSM and picks its own encoding,
// so both files would synthesise to (nearly) the same netlist and
// the comparison asked for in (f) would be meaningless.
//=====

module fsm_onehot (
    input wire clk,
    input wire rst, // synchronous, active high -> S1
    input wire [1:0] in,
    output reg [1:0] out
);

// ----- one-hot state codes -----
localparam [4:0] S1 = 5'b00001,
                 S2 = 5'b00010,
                 S3 = 5'b00100,
                 S4 = 5'b01000,
                 S5 = 5'b10000;

(* fsm_encoding = "none" *) reg [4:0] cur_state;
reg [4:0] next_state;

// ----- state register -----
always @(posedge clk) begin
    if (rst)
        cur_state <= S1;
    else
        cur_state <= next_state;
end

// ----- next state + output logic -----
always @(*) begin
    next_state = S1;
    out = 2'b00;

    case (cur_state)

        S1: case (in)
            2'b00: begin next_state = S1; out = 2'b01; end
            2'b01: begin next_state = S1; out = 2'b00; end
            2'b10: begin next_state = S2; out = 2'b01; end
            2'b11: begin next_state = S3; out = 2'b00; end
        endcase

        S2: case (in)
            2'b00: begin next_state = S1; out = 2'b11; end
            2'b01: begin next_state = S3; out = 2'b11; end
            2'b10: begin next_state = S2; out = 2'b10; end
            2'b11: begin next_state = S4; out = 2'b10; end
        endcase

        S3: case (in)
            2'b00: begin next_state = S1; out = 2'b00; end
            2'b01: begin next_state = S5; out = 2'b01; end
            2'b10: begin next_state = S3; out = 2'b10; end
            2'b11: begin next_state = S4; out = 2'b00; end
        endcase

        S4: case (in)
            2'b00: begin next_state = S3; out = 2'b11; end
            2'b01: begin next_state = S5; out = 2'b01; end
            2'b10: begin next_state = S5; out = 2'b01; end
        endcase
    endcase
end
```

```

        2'b11: begin next_state = S4; out = 2'b11; end
    endcase

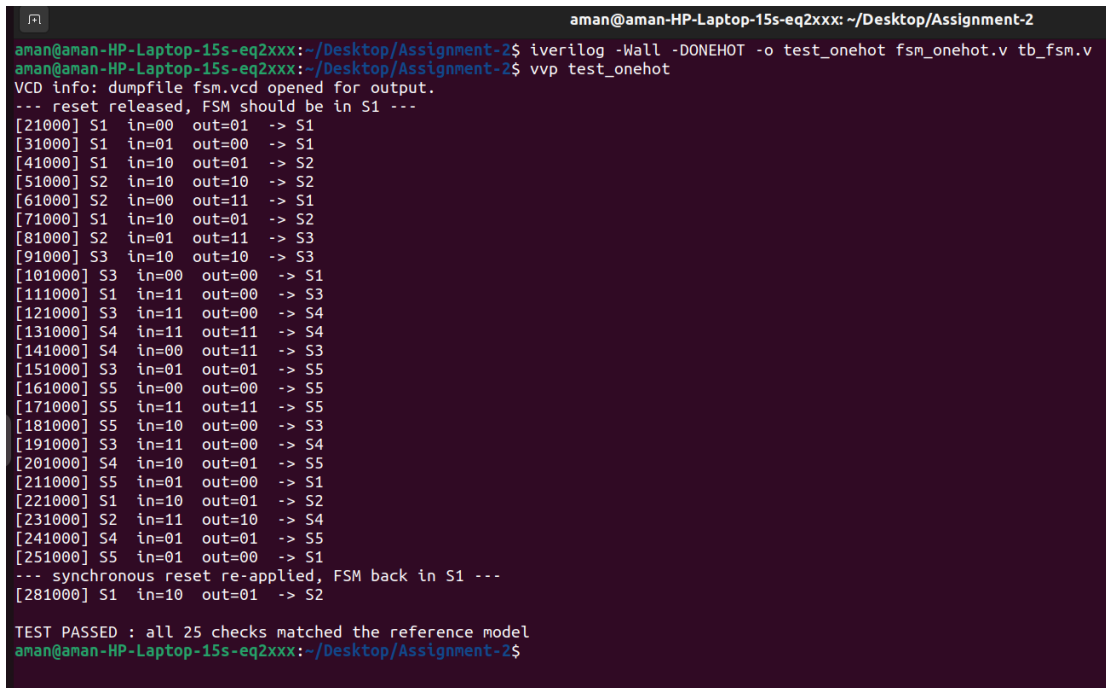
    S5: case (in)
        2'b00: begin next_state = S5; out = 2'b00; end
        2'b01: begin next_state = S1; out = 2'b00; end
        2'b10: begin next_state = S3; out = 2'b00; end
        2'b11: begin next_state = S5; out = 2'b11; end
    endcase

    default: begin next_state = S1; out = 2'b00; end
endcase
end
endmodule

```

Functional equivalence

Recompiling the same testbench with `-DONEHOT` gives an identical log and TEST PASSED on all 25 vectors, so only the encoding changed and any netlist difference is due to encoding alone.



```

aman@aman-HP-Laptop-15s-eq2xxx: ~/Desktop/Assignment-2
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ iverilog -Wall -DONEHOT -o test_onehot fsm_onehot.v tb_fsm.v
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$ vvp test_onehot
VCD info: dumpfile fsm.vcd opened for output.
--- reset released, FSM should be in S1 ---
[21000] S1 in=00 out=01 -> S1
[31000] S1 in=01 out=00 -> S1
[41000] S1 in=10 out=01 -> S2
[51000] S2 in=10 out=10 -> S2
[61000] S2 in=00 out=11 -> S1
[71000] S1 in=10 out=01 -> S2
[81000] S2 in=01 out=11 -> S3
[91000] S3 in=10 out=10 -> S3
[101000] S3 in=00 out=00 -> S1
[111000] S1 in=11 out=00 -> S3
[121000] S3 in=11 out=00 -> S4
[131000] S4 in=11 out=11 -> S4
[141000] S4 in=00 out=11 -> S3
[151000] S3 in=01 out=01 -> S5
[161000] S5 in=00 out=00 -> S5
[171000] S5 in=11 out=11 -> S5
[181000] S5 in=10 out=00 -> S3
[191000] S3 in=11 out=00 -> S4
[201000] S4 in=10 out=01 -> S5
[211000] S5 in=01 out=00 -> S1
[221000] S1 in=10 out=01 -> S2
[231000] S2 in=11 out=10 -> S4
[241000] S4 in=01 out=01 -> S5
[251000] S5 in=01 out=00 -> S1
--- synchronous reset re-applied, FSM back in S1 ---
[281000] S1 in=10 out=01 -> S2

TEST PASSED : all 25 checks matched the reference model
aman@aman-HP-Laptop-15s-eq2xxx:~/Desktop/Assignment-2$

```

Figure 10: Simulation of the one-hot design using the same testbench compiled with `-DONEHOT`. The log is identical to Figure 1.

Synthesised netlist — fsm_onehot_netlist_lib.v

Synthesised with the same script, passes and library, changing only the file and module names, so the comparison is like for like.

```

/* Generated by Yosys 0.9 (git sha1 1979e0b) */

module fsm_onehot(clk, rst, in, out);
    wire [4:0] _000_;
    wire _001_; wire _002_; wire _003_; wire _004_; wire _005_; wire _006_;
    wire _007_; wire _008_; wire _009_; wire _010_; wire _011_; wire _012_;
    wire _013_; wire _014_; wire _015_; wire _016_; wire _017_; wire _018_;
    wire _019_; wire _020_; wire _021_; wire _022_; wire _023_; wire _024_;
    wire _025_; wire _026_; wire _027_; wire _028_; wire _029_; wire _030_;
    wire _031_; wire _032_; wire _033_; wire _034_; wire _035_; wire _036_;
    wire _037_; wire _038_; wire _039_; wire _040_; wire _041_; wire _042_;
    wire _043_; wire _044_; wire _045_; wire _046_; wire _047_; wire _048_;
    wire _049_; wire _050_; wire _051_; wire _052_; wire _053_; wire _054_;
    wire _055_; wire _056_; wire _057_; wire _058_; wire _059_; wire _060_;
    wire _061_; wire _062_;
    input clk;
    wire [4:0] cur_state;
    input [1:0] in;
    output [1:0] out;
    input rst;

    // ----- shared input / state decode -----
    INVX1 _063_ ( .A(in[0]), .Y(_001_) );
    INVX1 _064_ ( .A(in[1]), .Y(_002_) );
    INVX1 _065_ ( .A(cur_state[2]), .Y(_003_) );
    INVX1 _066_ ( .A(cur_state[4]), .Y(_004_) );
    INVX1 _067_ ( .A(cur_state[0]), .Y(_005_) );
    INVX1 _068_ ( .A(cur_state[1]), .Y(_006_) );

```

```

NOR2X1 _069 ( .A(cur_state[0]), .B(cur_state[1]), .Y(_007_) );
AND2X1 _070 ( .A(_004_), .B(_007_), .Y(_008_) );
NOR2X1 _071 ( .A(_003_), .B(cur_state[3]), .Y(_009_) );
NAND2X1 _072 ( .A(_008_), .B(_009_), .Y(_010_) );
OR2X1 _073 ( .A(cur_state[2]), .B(cur_state[3]), .Y(_011_) );
NOR2X1 _074 ( .A(cur_state[4]), .B(_011_), .Y(_012_) );
NOR2X1 _075 ( .A(cur_state[0]), .B(_006_), .Y(_013_) );
NAND2X1 _076 ( .A(_012_), .B(_013_), .Y(_014_) );
INVX1 _077 ( .A(_014_), .Y(_015_) );
NAND2X1 _078 ( .A(_010_), .B(_014_), .Y(_016_) );
NAND2X1 _079 ( .A(cur_state[3]), .B(_008_), .Y(_017_) );
NOR2X1 _080 ( .A(cur_state[2]), .B(_017_), .Y(_018_) );
OR2X1 _081 ( .A(_016_), .B(_018_), .Y(_019_) );
NAND2X1 _082 ( .A(cur_state[4]), .B(_007_), .Y(_020_) );
OR2X1 _083 ( .A(_011_), .B(_020_), .Y(_021_) );
NOR2X1 _084 ( .A(_005_), .B(cur_state[1]), .Y(_022_) );
NAND2X1 _085 ( .A(_012_), .B(_022_), .Y(_023_) );
NAND2X1 _086 ( .A(_021_), .B(_023_), .Y(_024_) );
NOR2X1 _087 ( .A(_019_), .B(_024_), .Y(_025_) );
NOR2X1 _088 ( .A(in[0]), .B(in[1]), .Y(_026_) );

// ----- next-state bit 0 (state S1) -----
NAND2X1 _089 ( .A(_016_), .B(_026_), .Y(_027_) );
NAND2X1 _090 ( .A(in[0]), .B(_002_), .Y(_028_) );
NOR2X1 _091 ( .A(_021_), .B(_028_), .Y(_029_) );
OR2X1 _092 ( .A(in[1]), .B(_023_), .Y(_030_) );
NOR2X1 _093 ( .A(rst), .B(_029_), .Y(_031_) );
NAND2X1 _094 ( .A(_027_), .B(_030_), .Y(_032_) );
NOR2X1 _095 ( .A(_025_), .B(_032_), .Y(_033_) );
NAND2X1 _096 ( .A(_031_), .B(_033_), .Y(_000_[0]) );

// ----- next-state bit 1 (state S2) -----
NAND2X1 _097 ( .A(_014_), .B(_023_), .Y(_034_) );
NAND2X1 _098 ( .A(_001_), .B(in[1]), .Y(_035_) );
NOR2X1 _099 ( .A(rst), .B(_035_), .Y(_036_) );
AND2X1 _100 ( .A(_034_), .B(_036_), .Y(_000_[1]) );

// ----- next-state bit 2 (state S3) -----
NOR2X1 _101 ( .A(_014_), .B(_028_), .Y(_037_) );
NOR2X1 _102 ( .A(_001_), .B(_002_), .Y(_038_) );
NAND2X1 _103 ( .A(in[0]), .B(in[1]), .Y(_039_) );
NOR2X1 _104 ( .A(_023_), .B(_039_), .Y(_040_) );
AND2X1 _105 ( .A(_018_), .B(_026_), .Y(_041_) );
NOR2X1 _106 ( .A(_010_), .B(_035_), .Y(_042_) );
NOR2X1 _107 ( .A(_021_), .B(_035_), .Y(_043_) );
NOR2X1 _108 ( .A(_037_), .B(_042_), .Y(_044_) );
NOR2X1 _109 ( .A(_040_), .B(_043_), .Y(_045_) );
NAND2X1 _110 ( .A(_044_), .B(_045_), .Y(_046_) );
NOR2X1 _111 ( .A(_041_), .B(_046_), .Y(_047_) );
NOR2X1 _112 ( .A(rst), .B(_047_), .Y(_000_[2]) );

// ----- next-state bit 3 (state S4) -----
NAND2X1 _113 ( .A(_019_), .B(_038_), .Y(_048_) );
NOR2X1 _114 ( .A(rst), .B(_048_), .Y(_000_[3]) );

// ----- next-state bit 4 (state S5) -----
NOR2X1 _115 ( .A(_026_), .B(_038_), .Y(_049_) );
NAND2X1 _116 ( .A(_018_), .B(_049_), .Y(_050_) );
NOR2X1 _117 ( .A(_010_), .B(_028_), .Y(_051_) );
NOR2X1 _118 ( .A(_021_), .B(_049_), .Y(_052_) );
NOR2X1 _119 ( .A(_051_), .B(_052_), .Y(_053_) );
AND2X1 _120 ( .A(_050_), .B(_053_), .Y(_054_) );
NOR2X1 _121 ( .A(rst), .B(_054_), .Y(_000_[4]) );

// ----- Mealy output logic -----
NOR2X1 _122 ( .A(_021_), .B(_039_), .Y(_055_) );
NOR2X1 _123 ( .A(_018_), .B(_055_), .Y(_056_) );
NOR2X1 _124 ( .A(in[0]), .B(_023_), .Y(_057_) );
NOR2X1 _125 ( .A(in[1]), .B(_014_), .Y(_058_) );
NOR2X1 _126 ( .A(_051_), .B(_058_), .Y(_059_) );
NAND2X1 _127 ( .A(_056_), .B(_059_), .Y(_060_) );
OR2X1 _128 ( .A(_057_), .B(_060_), .Y(out[0]) );
OR2X1 _129 ( .A(_049_), .B(_056_), .Y(_061_) );
NOR2X1 _130 ( .A(_015_), .B(_042_), .Y(_062_) );
NAND2X1 _131 ( .A(_061_), .B(_062_), .Y(out[1]) );

// ----- state register: 5 library flip-flops -----
DFFPOSX1 _132 ( .CK(clk), .D(_000_[0]), .Q(cur_state[0]) );
DFFPOSX1 _133 ( .CK(clk), .D(_000_[1]), .Q(cur_state[1]) );
DFFPOSX1 _134 ( .CK(clk), .D(_000_[2]), .Q(cur_state[2]) );
DFFPOSX1 _135 ( .CK(clk), .D(_000_[3]), .Q(cur_state[3]) );
DFFPOSX1 _136 ( .CK(clk), .D(_000_[4]), .Q(cur_state[4]) );
endmodule

```

Reading the netlist

Five DFFPOSX1 instances now, and five next-state cones. The reset is asymmetric, and tracing it checks the encoding is right. Bits 2, 3 and 4 each end in a NOR2X1 taking rst, so reset drives them to 0. Bit 1 ends in AND2X1 _100_, whose input _036_ is NOR2X1(rst, _035_) and goes to 0 under reset. Bit 0 ends in NAND2X1 _096_, whose input _031_ is also NOR2X1(rst, _029_) and goes to 0 — but a NAND with a 0 input gives 1. The register therefore resets to 00001, which is S1.

The arrival logic is visible directly. Gates _115_ to _121_ form the cone for bit 4, combining the arcs into S5 — S3 on 01, S4 on 01 and 10, and the two self-loops. Five such trees are needed rather than three, each ORing together every incoming arc.

Comparison

Metric	Gray (3-bit)	One-hot (5-bit)	Change
Chip area	147.0	201.0	+37%
Flip-flops (DFFPOSX1)	3	5	+2
Flip-flop area (12 each)	36.0	60.0	+67%
Combinational area	111.0	141.0	+27%
Total cells	59	74	+25%
NAND2X1 / NOR2X1	17 / 24	19 / 33	increase
AND2X1 / OR2X1 / INVX1	2 / 5 / 8	4 / 6 / 7	increase

Observations

1. Area increases. The one-hot design occupies 201.0 units against 147.0, up 37 percent. Both parts grow: flip-flops from 36.0 to 60.0 units, the cost of five state bits instead of three, and combinational logic from 111.0 to 141.0.
2. The combinational logic did not get simpler, against the usual expectation. One-hot is advocated because testing whether the machine is in a given state costs one wire, while a dense encoding needs a decode gate. That saving is real but small at three bits wide. Against it sits a cost independent of the decode: five next-state bits must be computed rather than three, each the OR of every arc arriving at that state. With 20 arcs over 5 destinations this arrival logic is substantial — NOR2X1 rises from 24 to 33.
3. Crossover point. One-hot improves with state count: a dense encoding needs N decoders of $\log_2 N$ inputs each, while one-hot needs none. Five states sits well below where this pays off, which is why gray wins on every metric here. With tens of states the result would reverse, and one-hot is the usual choice for large control FSMs.
4. Technology matters. On an FPGA every logic cell already contains a flip-flop and LUT depth sets the speed, so one-hot is the default. In an ASIC flow each flip-flop costs real area and leakage, favouring denser encodings unless timing demands otherwise. A richer library with AOI and OAI cells would also compress the one-hot arrival logic, a wide two-level AND-OR structure, narrowing the gap measured here.
5. Scope of the result. This is an area comparison. The usual argument for one-hot concerns critical path depth and clock frequency, which would need static timing analysis against the library timing tables to establish. So the conclusion is that for this machine, at this size, against this library, gray is smaller — not that one-hot is worse in general, and nothing about which runs faster.
6. Robustness. One-hot uses 5 of 32 register values, leaving 27 illegal states against 3 for gray, so a single-bit upset is far more likely to land in one — possibly with two bits hot at once. The default recovery branch therefore matters more here. Gray, meanwhile, limits simultaneous bit switching on transitions between adjacent codes, reducing switching noise and dynamic power.